

DATABASE INDEX

인덱스, 왜 빠른가

회원 1,000만 명 중, 이메일 하나를 찾는 방법



```
SELECT * FROM member WHERE email = 'xxx@xxx.com';
```

인덱스가 없다면 어떻게 될까?

회원 1,000만 명 테이블에서 이메일 하나를 조건 검색한다

```
SELECT * FROM member WHERE email = 'kim@example.com';
```

10,000,000

전체 회원 수 (rows)

0개

email 컬럼 인덱스

$O(n)$

탐색 시간복잡도

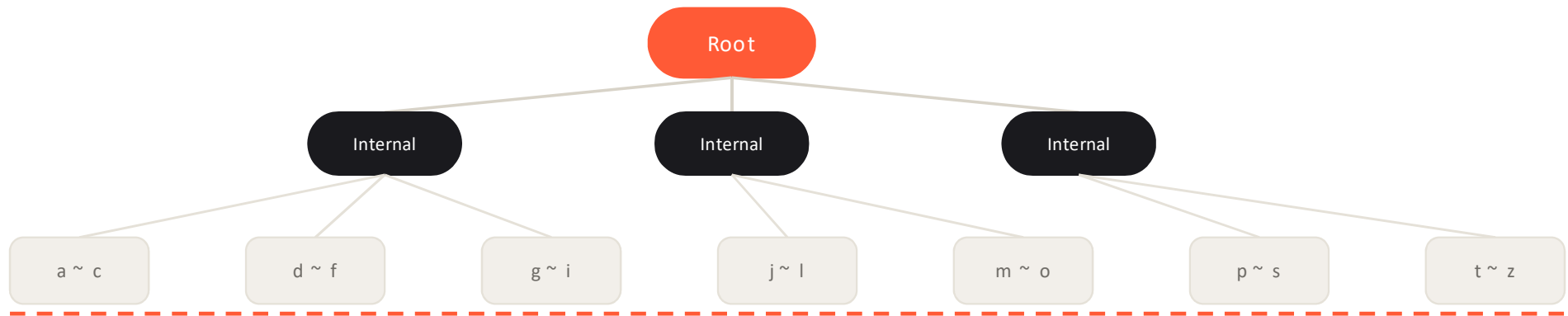
→ 조건에 맞는 행을 찾으려면, 1번 행부터 1,000만 번째 행까지 하나씩 다 확인해야 한다 (Full Table Scan)

EXPLAIN 결과

type: **ALL** rows: **10,000,000** Extra: Using where

email에 인덱스를 걸면? B+Tree

MySQL(InnoDB)의 인덱스는 대부분 B+Tree — 정렬을 유지하며 로그 시간에 탐색



email 알파벳 순으로 정렬되어 구간별 Leaf에 분산 저장 · Leaf끼리는 Linked List로 연결 → 범위 검색(LIKE, BETWEEN)이 빠른 이유

- 정렬 유지

삽입/삭제해도 항상 정렬 상태 유지

- $O(\log n)$ 탐색

1,000만 건이어도 트리 깊이는 3~4단계

- 범위 검색에 강함

Leaf 연결 구조로 순차 스캔 가능

EXPLAIN 으로 확인하는 변화

같은 쿼리, email 인덱스 유무에 따른 실행계획 비교

BEFORE — 인덱스 없음

type

ALL

rows (estimated)

10,000,000

Extra: Using where

AFTER — email UNIQUE 인덱스

type

const

rows (estimated)

1

Extra: -

email에 UNIQUE 인덱스가 있다면, 옵티마이저는 이 조건을 상수(const)처럼 취급해 인덱스에서 단 1건만 즉시 조회한다

EXPLAIN

type: ALL → const

rows 급감

UNIQUE 인덱스

클러스터드 vs 논클러스터드 인덱스

email 인덱스로 PK를 찾아도, SELECT *는 테이블 본체로 한 번 더 이동해야 한다

① 논클러스터드 인덱스 (email)

ahn@mail.com	PK 88
kim@mail.com	PK 210
yun@mail.com	PK 340

email 정렬 + PK 포인터

① 인덱스 탐색

② 클러스터드 인덱스 (PK 순 실제 테이블)

PK 208
PK 209
PK 210 · kim@mail.com
PK 211
PK 212

PK 순서로 실제 row 데이터가 저장됨 (랜덤 I/O)

클러스터드 vs 논클러스터드

구조

PK 순 정렬 = 테이블 자체 · 별도 구조 + PK 포인터

테이블당 개수

1개 (PK 기본) · 여러 개 생성 가능

SELECT *

바로 조회 · PK로 한 번 더 조회

→ email 인덱스가 있어도 SELECT *는 인덱스 탐색 + 테이블 랜덤 I/O, 두 단계가 필요하다

Clustered Index

Non-Clustered Index

Random I/O

PK Lookup

커버링 인덱스: 테이블까지 안 가도 된다?

필요한 컬럼이 인덱스 안에만 있다면, 테이블 접근 자체가 사라진다

SELECT * (일반 조회)

- 1 email 인덱스 탐색
- 2 PK로 테이블 이동
- 3 row 전체 반환

SELECT email (커버링)

- 1 email 인덱스 탐색
- 2 인덱스에서 바로 반환
- 3 테이블 접근 없음

같은 이메일 인덱스라도, 필요한 컬럼만 물어보면(email만) 인덱스 안에서 답이 끝난다 — 테이블 접근(랜덤 I/O) 자체가 사라진다

1회

인덱스 탐색만 수행

0회

테이블 랜덤 I/O

SELECT email

필요한 컬럼만 조회

복합 인덱스: 컬럼 순서가 왜 중요할까?

name과 email을 함께 조회한다면, 인덱스는 어떤 순서로 정렬해야 할까

idx_name_email (name, email)

kim ahn@mail.com

kim kim@mail.com

kim yun@mail.com

lee ban@mail.com

park cho@mail.com

이 인덱스, 조건에 따라 탈 수 있을까?

- ✓ WHERE name = 'kim'
앞쪽 컬럼(name)만 사용 — 인덱스 탐
- ✓ WHERE name = 'kim' AND email = '...'
두 컬럼 모두 활용 — 가장 빠름
- ✗ WHERE email = '...'
선두 컬럼인 name 조건이 없어 일반적인 인덱스 범위 탐색에는 활용하기 어렵다

Leftmost Prefix Rule — 복합 인덱스는 맨 앞 컬럼부터 순서대로만 활용 가능. 기준은 조건이 쓰인 순서가 아니라 인덱스를 정의한 컬럼 순서다

Composite Index

Leftmost Prefix

idx_name_email

인덱스 컬럼을 가공하면 무슨 일이 벌어질까?

email에 함수를 씌우는 순간, 정렬된 인덱스를 더 이상 못 믿는다

인덱스가 '못 믿는' 순간

- 인덱스는 컬럼의 '원본 값'을 정렬해서 저장한다
- LOWER(), YEAR() 가공 시 결과값은 인덱스에 없다
- LIKE '%kim'처럼 뒤에서부터 매칭해도 정렬을 못 쓴다
- 가공한 값은 인덱스 정렬 순서와 안 맞아 탐색이 어렵다

이 조건, 인덱스를 탈 수 있을까?

- ✓ WHERE email = 'kim@mail.com'
원본 값 그대로 비교 — 인덱스 탐
- ✗ WHERE LOWER(email) = 'kim@mail.com'
LOWER() 결과값은 인덱스에 없음 — Full Scan
- ✓ WHERE email LIKE 'kim%'
앞부분 고정, 전방 일치 — 정렬 순서와 일치, 인덱스 탐
- ✗ WHERE email LIKE '%kim%'
선행 와일드카드로 탐색 시작 범위를 좁힐 수 없다

그래도 꼭 가공해서 조회해야 한다면? — 함수 기반 인덱스(Functional Index)를 별도로 고려할 수 있다

Full Scan 유발

전방 일치

LOWER()

함수 기반 인덱스

인덱스, 언제 걸어야 할까?

무조건 걸기보다, 실무에서는 아래 4가지를 먼저 확인한다

1 조회 조건에 자주 쓰이는가

WHERE · JOIN · ORDER BY · GROUP BY에 실제로 등장하는 컬럼인지 먼저 확인

2 카디널리티가 충분한가

성별처럼 값 종류가 적은 컬럼은 인덱스를 걸어도 효과가 낮다

3 쓰기 빈도 대비 이득이 큰가

INSERT/UPDATE가 잦은 테이블이면 인덱스 페이지 갱신·분할 비용을 함께 고려

4 이미 있는 인덱스와 겹치지 않는가

복합 인덱스 앞부분과 중복되는 단일 컬럼 인덱스는 대부분 불필요

체크리스트 개수가 아니라, 실제 쿼리 패턴과 인덱스 추가 전후 EXPLAIN 비교로 검증한다

Query Pattern

Cardinality

Write Cost

Index Overlap

그럼 인덱스는 많을수록 좋을까?

조회는 빨라지지만, 그 대가는 어디로 갈까

● 조회 (SELECT)

빨라진다 — $O(n) \rightarrow O(\log n)$

● 쓰기 (INSERT/UPDATE/DELETE)

느려진다 — 관련 인덱스마다 페이지 갱신·분할 비용 발생

● 낮은 카디널리티 컬럼

효과 적음 — 예: 성별처럼 값 종류가 적은 컬럼

결국 인덱스 설계는 “ 쿼리 패턴 ” 과 “ 카디널리티 ” 를 보고 결정하는 것

회원 1,000만 명, email 인덱스 하나로 $O(n)$ 을 $O(\log n)$ 으로 줄였지만 — 모든 컬럼에 걸 이유는 없다.